

exploracao-beforeIT-pt4

September 16, 2026

1 Introdução

Nesta seção final da exploração, vamos analisar os resultados da alteração feita ao introduzir a transferência de renda da população trabalhadora para os donos das empresas no modelo como proxy de um mercado de apostas.

Para fazer isto, vamos: 1. Entender como foi feita a transferência de renda. 2. Aprofundar na análise do impacto das transfência do PIB. 3. Procurar explicar os mecanismos deste impacto.

1.1 Transferências do mercado de apostas

O mercado de apostas foi introduzido como um mecanismo estilizado de transferência de renda entre famílias. Nesta versão, não existem casas de apostas, produção de serviços de apostas, probabilidades de vitória ou tributação específica, o mecanismo representa uma transferência recorrente de trabalhadores para proprietários de empresas. A implementação detalhada do mecanismo aparece na **Parte 2, Seção 2.37 — Transferências do mercado de apostas**.

1.1.1 Participantes e parâmetros

Sejam:

- $g \in [0, 1]$ a parcela da renda destinada às apostas, armazenada em `gambling_income_share`;
- G_A o conjunto fixo de trabalhadores ativos selecionados;
- G_N o conjunto fixo de trabalhadores inativos selecionados;
- R o conjunto de proprietários de empresas que recebem os recursos;
- $Y_{h,t}^{A,0}$ e $Y_{h,t}^{N,0}$ as rendas dos trabalhadores antes das apostas;
- $Y_{i,t}^{O,0}$ a renda dos proprietários antes dos recebimentos.

Na configuração inicial, aproximadamente 10% dos trabalhadores são selecionados como apostadores e 10% dos proprietários como destinatários. Os mesmos agentes permanecem nessas funções durante toda a simulação. No cenário com apostas, utiliza-se $g = 0,02$; no cenário de referência, $g = 0$.

Abaixo temos um diagrama que ilustra o impacto da alteração na simulação:

flowchart TD

```
subgraph MT["1. Mercado de trabalho"]
    direction LR
    A["Demanda de trabalho<br/>das empresas"]
    B["Contratações<br/>e demissões"]
    C["Emprego, salários<br/>e produção"]
end
```

```

    A --> B --> C
end

C --> D["Rendas esperadas<br/>das famílias"]

subgraph OP["2. Formação dos orçamentos"]
    direction LR
    E["Transferência esperada<br/>das apostas"]
    F["Renda esperada<br/>após as apostas"]
    G["Consumo e investimento<br/>desejados"]

    E --> F --> G
end

D --> E

subgraph MB["3. Mercado de bens"]
    direction LR
    H["Pareamento entre compradores<br/>e fornecedores"]
    I["Compras e vendas<br/>realizadas"]

    H --> I
end

G --> H
I --> J["Lucros e rendas<br/>realizados"]
J --> K["Transferência efetiva<br/>das apostas"]
K --> L["Depósitos e balanços<br/>atualizados"]

L -. "próximo período" .-> A

classDef trabalho fill:#e8f1fb,stroke:#3976a8,color:#172b3a;
classDef apostas fill:#fff3d6,stroke:#c58a18,color:#3d2d0d;
classDef bens fill:#e7f5eb,stroke:#3b8c55,color:#173622;
classDef resultado fill:#f1ecf8,stroke:#79589f,color:#2f2040;

class A,B,C trabalho;
class E,F,K apostas;
class G,H,I bens;
class D,J,L resultado;

```

1.1.2 Valor transferido pelos trabalhadores

Cada trabalhador selecionado transfere uma proporção fixa de sua renda disponível não negativa:

$$B_{h,t}^A = \mathbf{1}(h \in G_A) g \max(0, Y_{h,t}^{A,0}),$$

$$B_{h,t}^N = \mathbf{1}(h \in G_N) g \max(0, Y_{h,t}^{N,0}).$$

O operador $\max(0, Y)$ impede que agentes com renda negativa realizem apostas. Trabalhadores não selecionados transferem zero. O desconto sobre as rendas esperadas é apresentado na **Parte 2, Seção 2.18 — Orçamento dos trabalhadores ativos** e na **Seção 2.19 — Orçamento das pessoas inativas**.

O volume total de apostas no período é:

$$B_t = \sum_h B_{h,t}^A + \sum_h B_{h,t}^N.$$

Esse valor é armazenado pelo modelo em `gambling_volume`.

1.1.3 Distribuição aos proprietários

O volume arrecadado é dividido igualmente entre os proprietários selecionados:

$$R_{i,t} = \begin{cases} \frac{B_t}{|R|}, & i \in R, \\ 0, & i \notin R. \end{cases}$$

A inclusão dessas receitas no orçamento esperado dos beneficiários é detalhada na **Parte 2, Seção 2.20 — Orçamento dos proprietários das empresas**. As rendas depois da transferência são:

$$Y_{h,t}^A = Y_{h,t}^{A,0} - B_{h,t}^A,$$

$$Y_{h,t}^N = Y_{h,t}^{N,0} - B_{h,t}^N,$$

$$Y_{i,t}^O = Y_{i,t}^{O,0} + R_{i,t}.$$

O proprietário do banco não participa do mecanismo, nem como apostador nem como destinatário. Seu orçamento permanece conforme a **Parte 2, Seção 2.21 — Orçamento do proprietário do banco**.

1.1.4 Conservação da renda

A transferência é de soma zero, pois todo o valor retirado dos trabalhadores é entregue aos proprietários:

$$\sum_h B_{h,t}^A + \sum_h B_{h,t}^N = \sum_i R_{i,t}.$$

Consequentemente, a renda agregada das famílias é preservada:

$$\sum_h Y_{h,t}^A + \sum_h Y_{h,t}^N + \sum_i Y_{i,t}^O = \sum_h Y_{h,t}^{A,0} + \sum_h Y_{h,t}^{N,0} + \sum_i Y_{i,t}^{O,0}.$$

A demonstração operacional dessa conservação encontra-se na **Parte 2, Seção 2.37**. A transferência não é tributada, não representa produção e não é adicionada diretamente ao PIB.

1.1.5 Incorporação no ciclo do modelo

O mecanismo aparece em dois momentos distintos:

1. **Formação dos orçamentos:** a transferência esperada é descontada da renda dos apostadores e adicionada à renda dos proprietários antes do cálculo das demandas desejadas, conforme as **Seções 2.18 a 2.20 da Parte 2**.
2. **Atualização contábil:** depois do cálculo das rendas realizadas nas **Seções 2.33 a 2.35 da Parte 2**, a transferência efetiva é executada pela rotina descrita na **Seção 2.37**, antes da atualização dos depósitos apresentada nas **Seções 2.38 a 2.40**.

A renda esperada após as apostas, $\tilde{Y}_{h,t}^e$, determina as demandas desejadas:

$$C_{h,t}^d = \frac{\psi \tilde{Y}_{h,t}^e}{1 + \tau_{VAT}}, \quad I_{h,t}^d = \frac{\psi_H \tilde{Y}_{h,t}^e}{1 + \tau_{CF}}.$$

Assim, as apostas competem com consumo, investimento residencial e poupança no orçamento dos apostadores. Para os proprietários beneficiados, os recebimentos ampliam essas mesmas possibilidades.

Como todos os tipos de família usam as mesmas propensões ψ e ψ_H , o efeito imediato sobre a demanda agregada tende a ser pequeno. O efeito de primeira ordem é distributivo pois apostadores perdem renda e depósitos, enquanto proprietários selecionados os acumulam. Efeitos sobre produção e PIB podem surgir apenas indiretamente, por meio das interações posteriores do modelo.

1.2 Impacto das transferências no PIB

O impacto da alteração no PIB simulado para o caso de uma transferência de \$ g=2\% \$ da renda e participação de 10% das famílias não donas de empresas foi mensurado através de 20 simulações comparadas, onde, para uma mesma seed de geração de números aleatórios, a transferência foi ativada ou não. Cada simulação rodou por 16 trimestres.

```
[16]: using CSV
      using DataFrames
      using Statistics
      using Plots
      using Distributions: TDist, ccdf
      using StatsPlots
      using Markdown
      using Printf
```

Abaixo temos o impacto do trabalho que mostra a média da evolução do PIB Real para as simulações com e sem as transferências, bem como seus desvios padrões.

Ao analisarmos o gráfico, verificamos que, apesar da média com apostas ativas ter sido maior, há uma grande intersecção entre os desvios padrões, o que indica que em muitos casos o PIB da simulação com transferências ativas foi menor.

```
[11]: gdp_path = first(path for path in (
    joinpath(pwd(), "paired-gdp-paths.csv"),
    joinpath(pwd(), "dba-studies", "gambling-market", "paired-gdp-paths.csv"),
) if isfile(path))

gdp_paths = CSV.read(gdp_path, DataFrame)
@assert nrow(gdp_paths) == 20 * 2 * 17

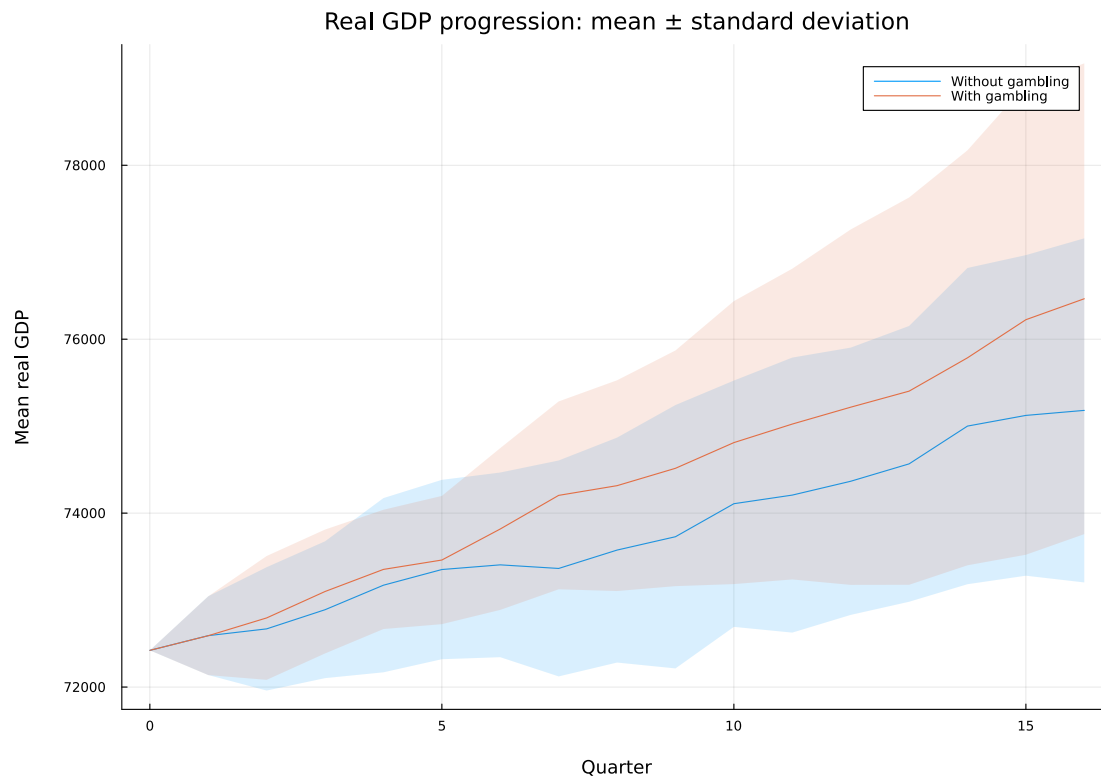
gdp_progression = combine(
    groupby(gdp_paths, [:scenario, :period]),
    :real_gdp => mean => :mean_real_gdp,
    :real_gdp => std => :std_real_gdp,
)
sort!(gdp_progression, [:scenario, :period])

baseline_gdp = filter(:scenario => ==("baseline"), gdp_progression)
gambling_gdp = filter(:scenario => ==("gambling"), gdp_progression)

plot(
    baseline_gdp.period,
    baseline_gdp.mean_real_gdp;
    ribbon = baseline_gdp.std_real_gdp,
    fillalpha = 0.15,
    label = "Without gambling",
    xlabel = "\nQuarter",
    ylabel = "Mean real GDP\n",
    title = "Real GDP progression: mean ± standard deviation",
    bottom_margin = 5Plots.mm,
    left_margin = 5Plots.mm,
    size = (1000,700)
)

plot!(
    gambling_gdp.period,
    gambling_gdp.mean_real_gdp;
    ribbon = gambling_gdp.std_real_gdp,
    fillalpha = 0.15,
    label = "With gambling",
)
```

[11]:



Para verificarmos isto de maneira mais precisa, vamos analisar os intervalos de confiança estatísticos de cada cenário de simulação e entender se há relevância estatística.

Abaixo temos uma tabela que compila o teste estatístico para as variáveis: - PIB ao final do período.
- Depósito das famílias. - Depósitos dos empresários. - Consumo agregado.

Podemos verificar, usando a distribuição Student t para aproximar a distribuição das médias, que nenhuma das variáveis tem impacto com significância estatística com intervalo de confiança de 95%.

```
[15]: results_path = first(path for path in (
    joinpath(pwd(), "paired-results.csv"),
    joinpath(pwd(), "dba-studies", "gambling-market", "paired-results.csv"),
) if isfile(path))

results = CSV.read(results_path, DataFrame)

tested_outcomes = [
    :final_real_gdp,
    :final_active_worker_deposits,
    :final_inactive_worker_deposits,
    :final_owner_deposits,
    :cumulative_household_consumption,
]
```

```

baseline = sort(filter(:scenario => ==("baseline"), results), :run)
gambling = sort(filter(:scenario => ==("gambling"), results), :run)

@assert baseline.run == gambling.run
@assert baseline.simulation_seed == gambling.simulation_seed

alpha = 0.05
number_of_tests = length(tested_outcomes)

significance = DataFrame(
  outcome = String[],
  mean_difference = Float64[],
  standard_error = Float64[],
  t_statistic = Float64[],
  degrees_of_freedom = Int[],
  p_value = Float64[],
  ci95_lower = Float64[],
  ci95_upper = Float64[],
)

for outcome in tested_outcomes
  differences = gambling[!, outcome] - baseline[!, outcome]

  n = length(differences)
  mean_difference = mean(differences)
  standard_error = std(differences) / sqrt(n)
  degrees_of_freedom = n - 1

  if iszero(standard_error)
    t_statistic = iszero(mean_difference) ? 0.0 : sign(mean_difference) * Inf
    p_value = iszero(mean_difference) ? 1.0 : 0.0
    margin = 0.0
  else
    distribution = TDist(degrees_of_freedom)
    t_statistic = mean_difference / standard_error
    p_value = 2 * ccdf(distribution, abs(t_statistic))
    critical_value = quantile(distribution, 1 - alpha / 2)
    margin = critical_value * standard_error
  end

  push!(
    significance,
    (
      string(outcome),
      mean_difference,

```

```

        standard_error,
        t_statistic,
        degrees_of_freedom,
        p_value,
        mean_difference - margin,
        mean_difference + margin,
    ),
)
end

display(significance)

```

	outcome	mean_difference	standard_error	t_statistic	
	String	Float64	Float64	Float64	
1	final_real_gdp	1283.77	626.303	2.04976	...
2	final_active_worker_deposits	360.003	509.248	0.70693	...
3	final_inactive_worker_deposits	196.432	269.199	0.72969	...
4	final_owner_deposits	262.782	217.767	1.20671	...
5	cumulative_household_consumption	682.53	3429.92	0.198993	...

Este resultado nos provoca a realizar mais simulações com variações na quantidade de recursos transferidos e na participação da população que aposta. Ao fazer isto, podemos verificar se há uma relação entre PIB e renda transferida nas apostas.

Para isto, rodamos as seguintes configurações combinadas: - Percentuais de apostas: - Percentuais de famílias apostadoras:

Essas configurações resultaram em 340 simulações que nos permitem comparar diferentes cenários do PIB para cada uma delas.

```

[17]: study_dir = isfile(joinpath(pwd(), "gambling-grid-period-trace.csv")) ?
      pwd() : joinpath(pwd(), "dba-studies", "gambling-market")
      trace_path = joinpath(study_dir, "gambling-grid-period-trace.csv")
      results_path = joinpath(study_dir, "gambling-grid-results.csv")
      summary_path = joinpath(study_dir, "gambling-grid-cell-summary.csv")
      @assert all(isfile, [trace_path, results_path, summary_path])

      grid_trace = CSV.read(trace_path, DataFrame)
      grid_results = CSV.read(results_path, DataFrame)
      grid_summary = CSV.read(summary_path, DataFrame)
      sort!(grid_trace, [:run, :scenario_id, :period])

      @assert nrow(grid_results) == 340
      @assert nrow(grid_trace) == 5_780
      @assert length(unique(grid_results.scenario_id)) == 17
      (simulacoes = nrow(grid_results), trajetorias = nrow(grid_trace), celulas = 16)

```

```

[17]: (simulacoes = 340, trajetorias = 5780, celulas = 16)

```